



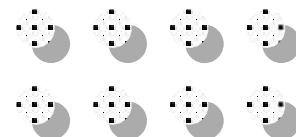
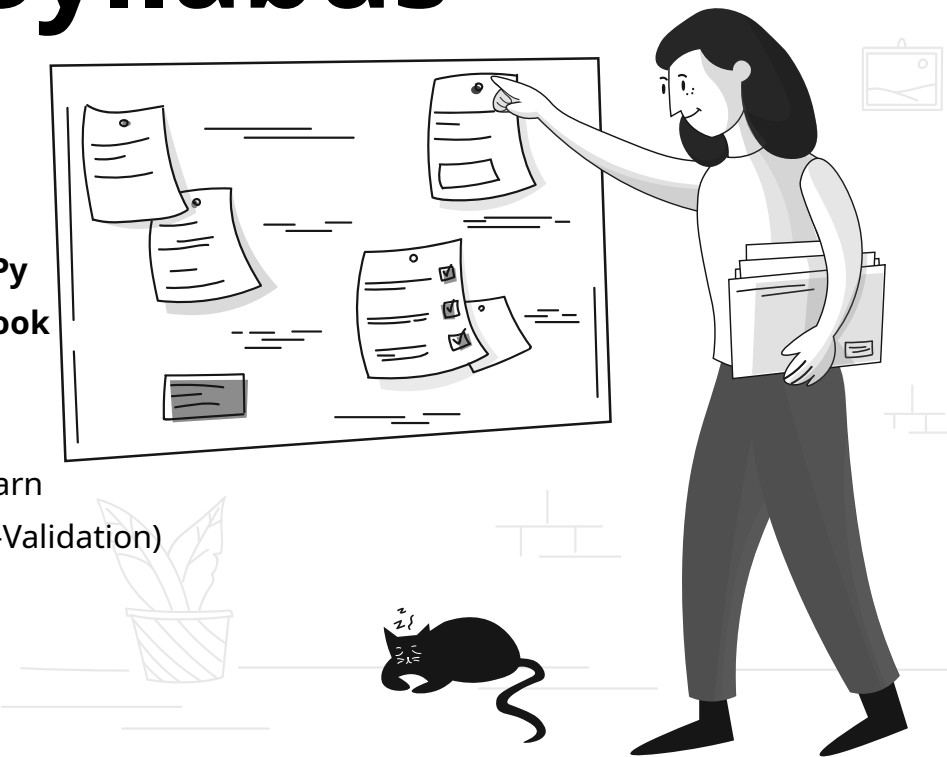
# Linear Algebra + NumPy

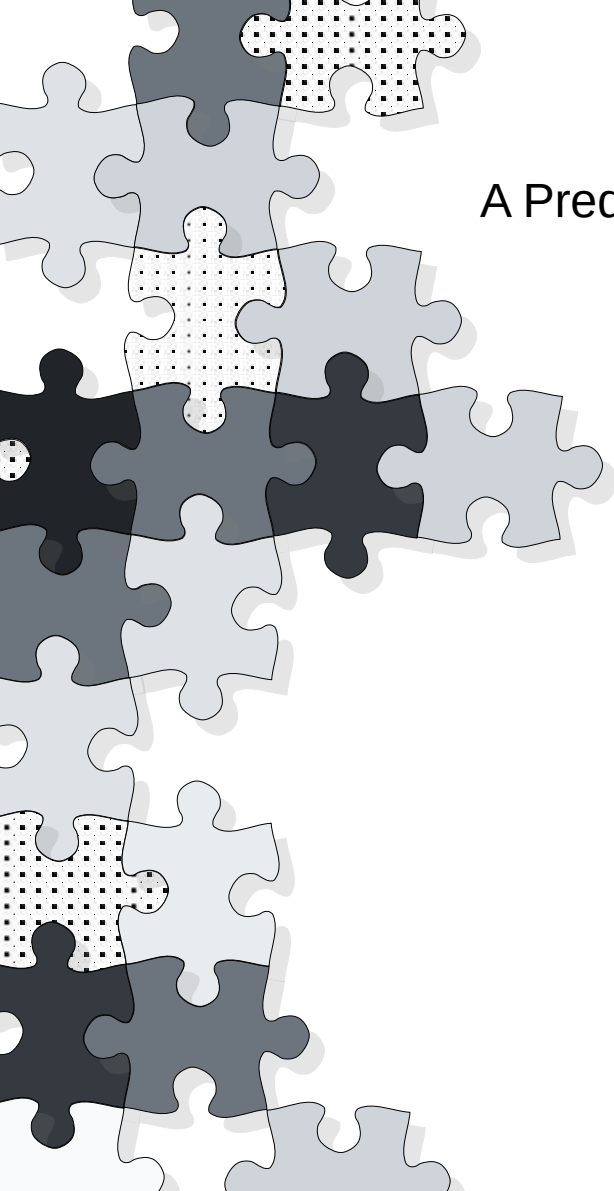
Eng | Mahmoud Tariq

09/08/2025

# Course Syllabus

- Intro to Machine Learning & Environment Setup
- **Foundations: Key Linear Algebra & Stats + Hands-on NumPy**
  - **Task: Solve session 2's 'task.ipynb' notebook**
- Data Handling & Visualization + Hands-on Pandas
- The ML Workflow + Linear Regression with Scikit-learn
- Classification Models (Logistic Regression, KNN) with Scikit-learn
- Model Evaluation & Improvement (Metrics, Overfitting, Cross-Validation)
- Mini-Project: Putting it all together with Scikit-learn
- Introduction to Neural Networks
- Building a Neural Network with a Framework
- Course Project Description + Where to Go From Here



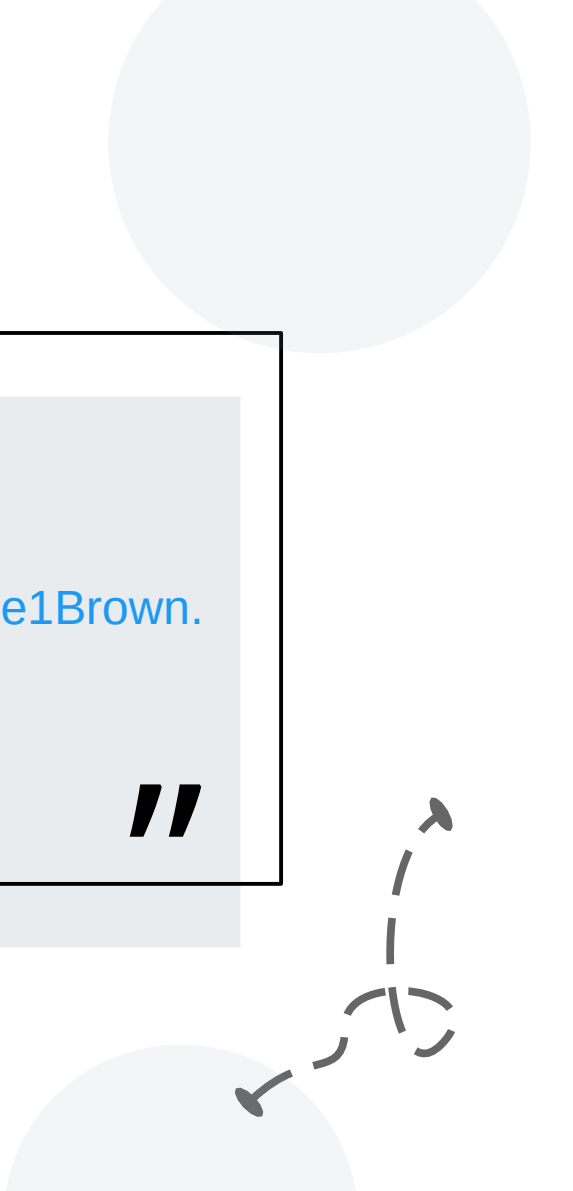


A Prerequisite to this session.

**//**

Essence of Linear Algebra By 3Blue1Brown.

**//**



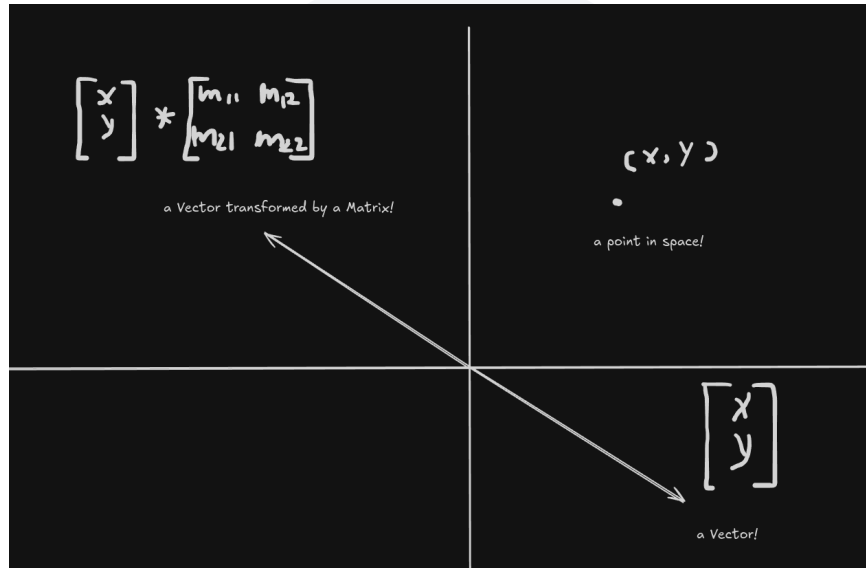
# Linear Algebra Concepts

- What is Linear Algebra?
- Why Linear Algebra?
- Tensors?



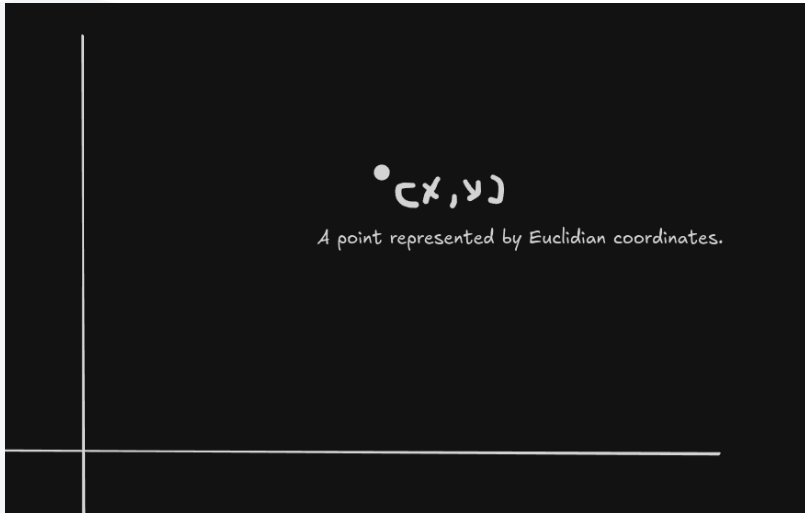
# Linear Algebra?

- Linear Algebra: A branch of Mathematics concerning linear equations, points in space, vectors, matrices and their interactions.



# Linear Algebra: Points?

- Points: points are dots in space placed at coordinates  $(x, y)$ , though they don't represent much alone, most concepts are built on top of them.

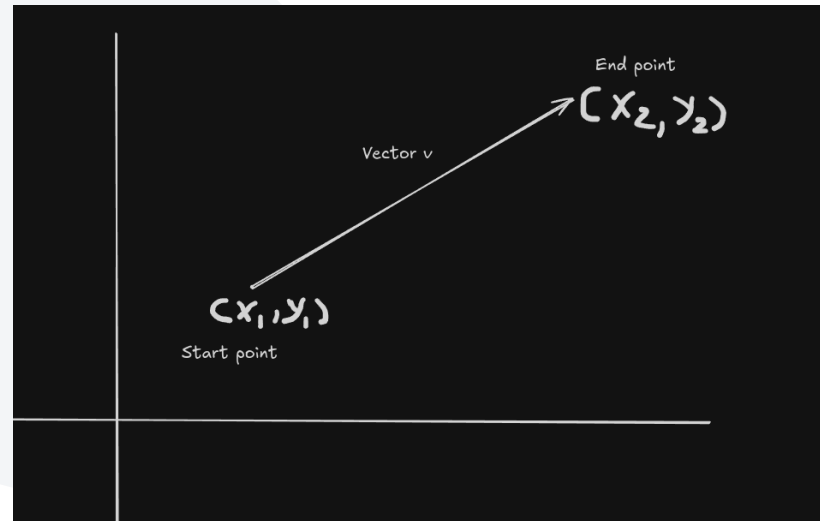


$(x, y)$

*A point represented by Euclidian coordinates.*

# Linear Algebra: Vectors?

- Vector: A vector in linear algebra is represented by a start point, an end point and an arrow from the start point pointing to the end point.
- Though the start point can be arbitrary, it is generally placed at the origin point  $(0, 0)$ .



# Linear Algebra: Vectors?

- Vector can be used in a lot of ways, for one they can represent a system of equations, as a vector is can be written as the following equation:

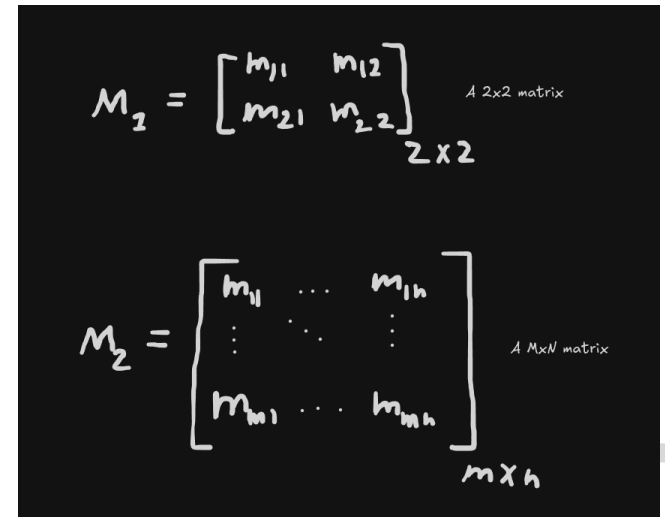
$$y = aX + b$$

- Where  $x$ ,  $y$  are the coordinates for the end point (assuming the start is the origin).
- And “ $a$ ” being the slope and “ $b$ ” is the intercept.



# Linear Algebra: Matrices?

- Matrix: A matrix is an ordered set of elements, sorted into rows and columns. A matrix's size is represented as  $M \times N$  where  $M$  is the number of rows and  $N$  is the number of columns.
- A matrix can be thought of as a transformation (check playlist for a visual representation).



Handwritten mathematical notation for matrices on a black background:

Top equation:  $M_2 = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix}$  with the text "A 2x2 matrix" and "2x2" below it.

Bottom equation:  $M_2 = \begin{bmatrix} m_{11} & \dots & m_{1h} \\ \vdots & \ddots & \vdots \\ m_{m1} & \dots & m_{mh} \end{bmatrix}$  with the text "A MxN matrix" and "m x h" below it.

# Linear Algebra: Matrices?

- Matrices have special attributes when it comes to their operations.
- For one addition/subtraction needs both matrices to be the same MxN size and the result will also have the same MxN size.

$$M_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}_{2 \times 2}, M_2 = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}_{2 \times 2}$$

$$M_3 = M_1 + M_2 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}_{2 \times 2} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}_{2 \times 2} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}_{2 \times 2}$$

# Linear Algebra: Matrices?

- Matrices multiplication is also special, there are two types: scalar and matrix-wise.
- First, Scalar: scalar matrix multiplication is simple, for each element in matrix “M” multiply it by scalar “a”.

$$M = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$a = 2$$

$$M_2 = M * a = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * 2$$

$$M_2 = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$$

# Linear Algebra: Matrices?

- Matrices multiplication is also special, there are two types: scalar and matrix-wise.
- First, Scalar: scalar matrix multiplication is simple, for each element in matrix “M” multiply it by scalar “a”.

$$M = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

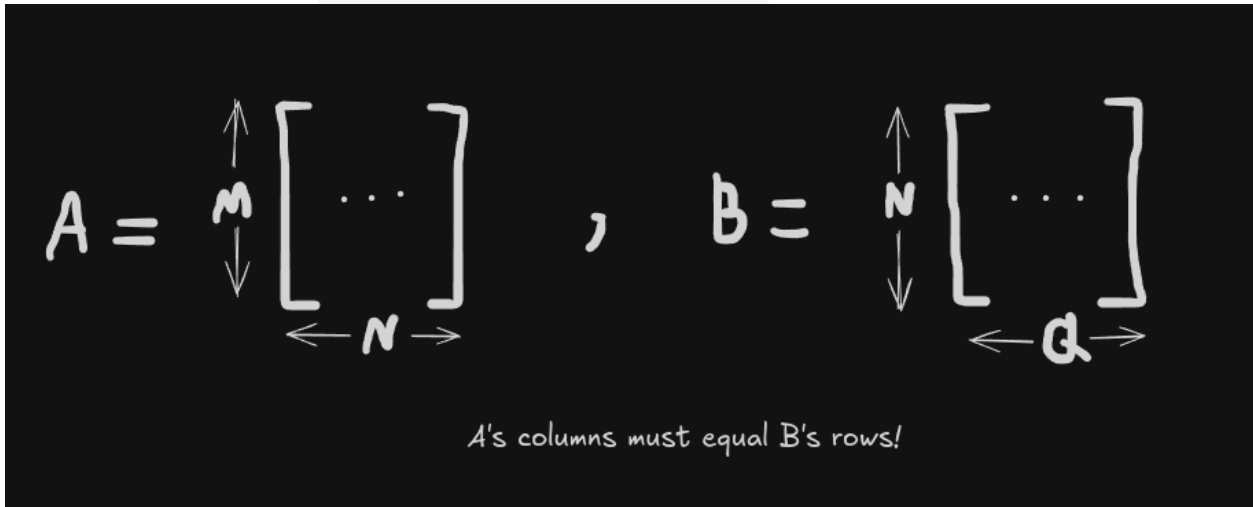
$$a = 2$$

$$M_2 = M * a = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * 2$$

$$M_2 = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$$

# Linear Algebra: Matrices?

- Second, matrix-wise multiplication. This operation requires that matrix “A” and matrix “B” must have the sizes:  $M \times N$  and  $N \times Q$  respectively, as in matrix “A” must have the same number of columns as “B” has rows (in this case A’s rows (M) and B’s columns (Q) can be arbitrary).


$$A = \begin{matrix} \updownarrow M \\ \left[ \begin{matrix} \dots \end{matrix} \right] \\ \leftarrow N \rightarrow \end{matrix}, \quad B = \begin{matrix} \updownarrow N \\ \left[ \begin{matrix} \dots \end{matrix} \right] \\ \leftarrow Q \rightarrow \end{matrix}$$

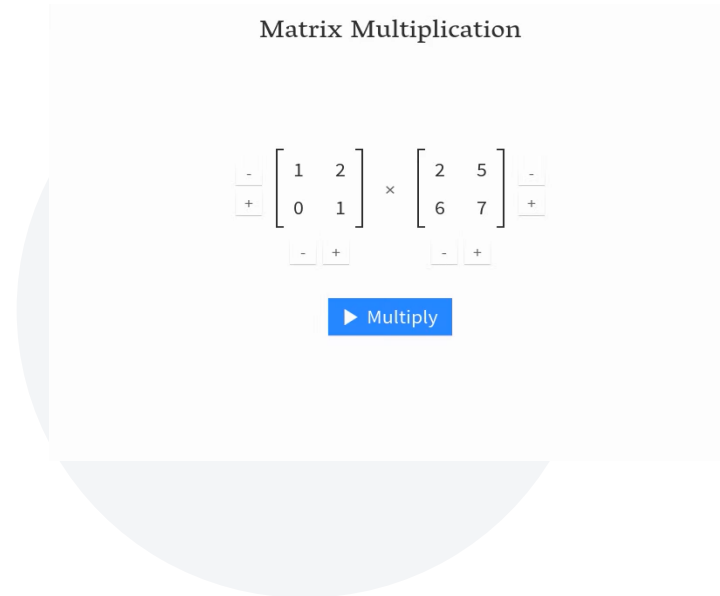
*A's columns must equal B's rows!*

# Linear Algebra: Matrices?

- Lastly the result will be of size  $M \times Q$ , “M” being A’s rows and “Q” being B’s columns.
- As for how to calculate it, it goes as follows:
  - For each column in “A” and each row of “B”:
  - Multiply each element from A’s column with the corresponding element from B’s row.
  - Add the result of the multiplications, and that’s the  $[i, j]$ ’th element in the resulting matrix (i being the A’s column number and j being B’s row number).

# Linear Algebra: Matrices?

- Check for animation: [Matrix Multiplication](#)



# Linear Algebra: Matrices?

- Final point about matrices is that vectors are technically a  $M \times 1$  (or  $1 \times N$ ) matrix.
- Another name for that is a column vector ( $M \times 1$ ) or a row vector ( $1 \times N$ ).
- Column vectors are usually the standard.

$$A = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}_{3 \times 1}, \quad B = [1 \ 2 \ 3]_{1 \times 3}$$



# Linear Algebra: Transformation?

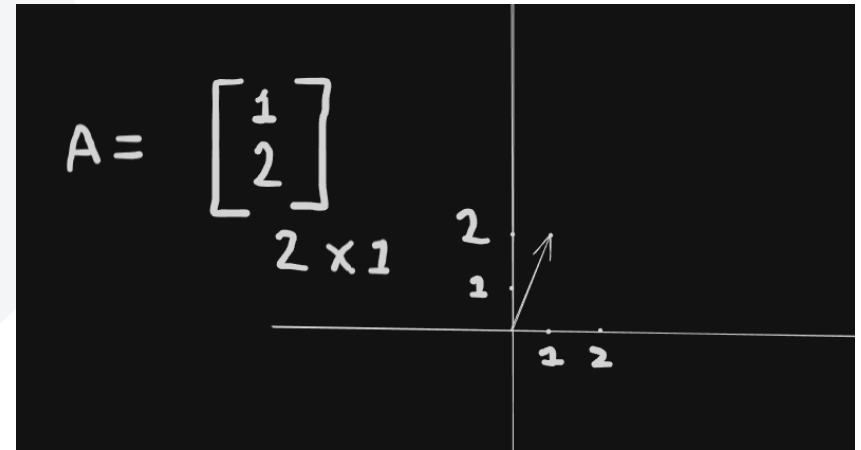
- Final point about matrices is that vectors are technically a  $M \times 1$  (or  $1 \times N$ ) matrix.
- Another name for that is a column vector ( $M \times 1$ ) or a row vector ( $1 \times N$ ).
- Column vectors are usually the standard, So we will continue the examples with column vectors only.

$$A = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}_{3 \times 1}$$

$$, \quad B = [1 \ 2 \ 3]_{1 \times 3}$$

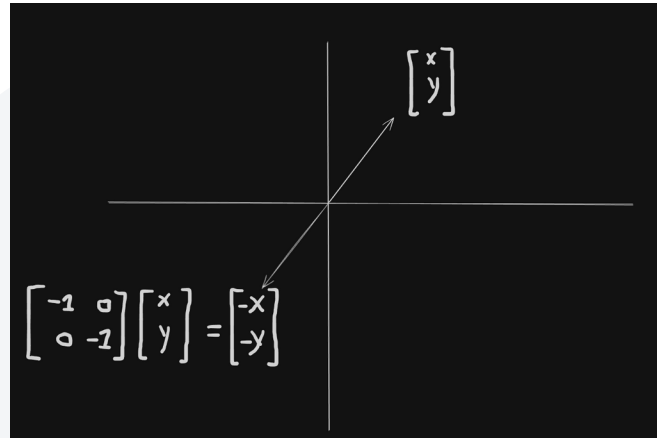
# Linear Algebra: Transformation?

- The number of rows a vector has represents its dimensionality (i.e. 2d, 3d, etc ...).
- The values in the column represent the coordinates for the end point (i.e. (x, y, z)).
- The starting point is always the origin.



# Linear Algebra: Transformation?

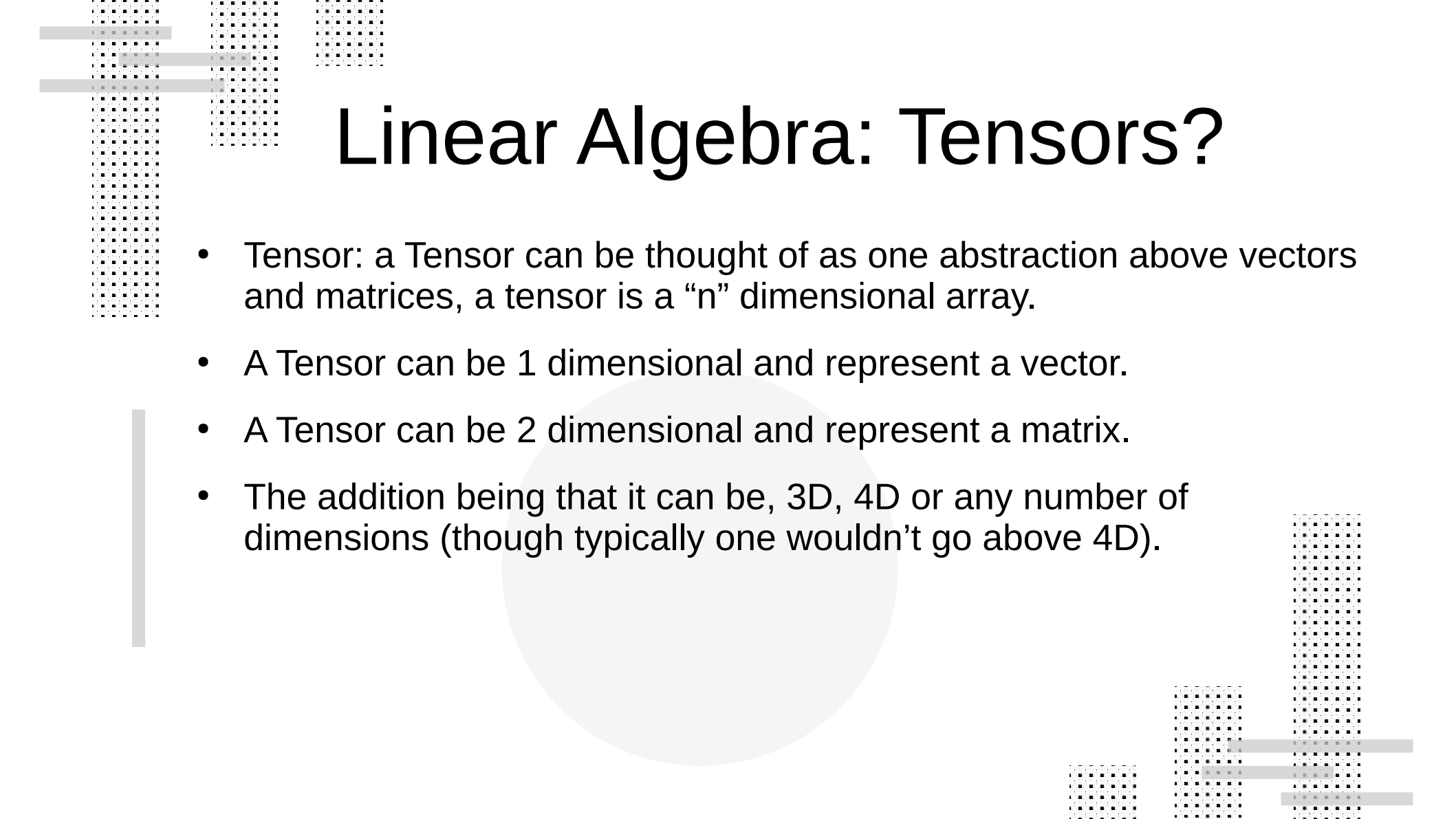
- This means that vectors and matrices can be multiplied, usually resulting in another column vector, but “Transformed”.
- Take for example:



- Here the vector was rotated 180 degrees.

# Linear Algebra: Why?

- Linear algebra is found in many places in AI/ML, some examples:
  - The weights in the models are represented in matrix transformations.
  - The datasets are represented as rows and columns (matrices, we will expand on this in the next session).
  - Data points are represented as vectors.
  - Whole models are represented as linear equations.
- This makes the interactions between these components easier as they can be represented as numbers and equations.



# Linear Algebra: Tensors?

- Tensor: a Tensor can be thought of as one abstraction above vectors and matrices, a tensor is a “n” dimensional array.
- A Tensor can be 1 dimensional and represent a vector.
- A Tensor can be 2 dimensional and represent a matrix.
- The addition being that it can be, 3D, 4D or any number of dimensions (though typically one wouldn't go above 4D).

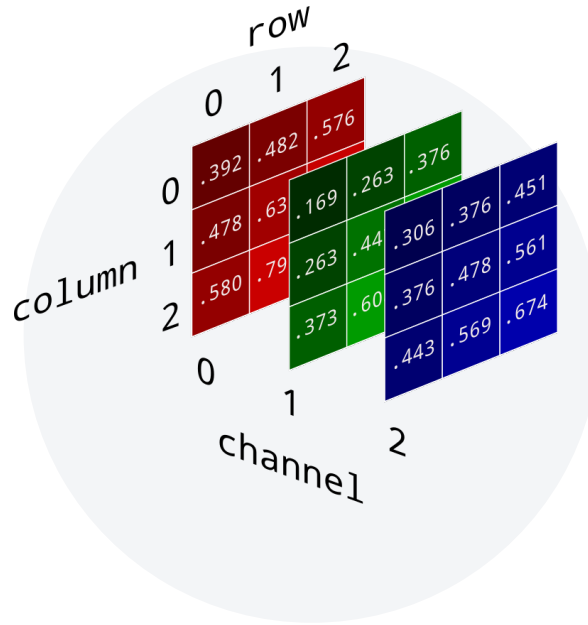
# Linear Algebra: Tensors?

- A example of a 2D tensor is a simple dataset with rows and columns:

| Id | SepalLeng | SepalWid | PetalLeng | PetalWid | Species     |
|----|-----------|----------|-----------|----------|-------------|
| 1  | 5.1       | 3.5      | 1.4       | 0.2      | Iris-setosa |
| 2  | 4.9       | 3        | 1.4       | 0.2      | Iris-setosa |
| 3  | 4.7       | 3.2      | 1.3       | 0.2      | Iris-setosa |
| 4  | 4.6       | 3.1      | 1.5       | 0.2      | Iris-setosa |
| 5  | 5         | 3.6      | 1.4       | 0.2      | Iris-setosa |
| 6  | 5.4       | 3.9      | 1.7       | 0.4      | Iris-setosa |
| 7  | 4.6       | 3.4      | 1.4       | 0.3      | Iris-setosa |
| 8  | 5         | 3.4      | 1.5       | 0.2      | Iris-setosa |
| 9  | 4.4       | 2.9      | 1.4       | 0.2      | Iris-setosa |
| 10 | 4.9       | 3.1      | 1.5       | 0.1      | Iris-setosa |
| 11 | 5.4       | 3.7      | 1.5       | 0.2      | Iris-setosa |
| 12 | 4.8       | 3.4      | 1.6       | 0.2      | Iris-setosa |
| 13 | 4.8       | 3        | 1.4       | 0.1      | Iris-setosa |

# Linear Algebra: Tensors?

- A example of a 3D tensor is an image:



# NumPy

- What is NumPy?
- NumPy Basics.





# What is NumPy?

- NumPy: is a popular python framework for scientific numerical calculation, it is the basis of many other libraries and integrates with most of ML frameworks (e.g. Pandas, sci-kit learn, tensorflow, etc...)
- We will use NumPy in general for simplifying calculations, creating sample data and basically anytime we need to deal with numbers.

# NumPy Basics: Import

- First ensure you activated your python environment (check previous slides for details).
- Then we import the library:

```
import numpy as np # importing NumPy as np is a standard and reduces code length by a bit
```

# NumPy Basics: ndarray

- Then we explore our most basic class in NumPy, the ndarray.
- From the name it sounds very similar to a Tensor, and they both serve the same purpose.
- We can convert from a normal python list to a ndarray with `np.array()`

```
normal_list = [1, 2, 3, 4, 5]
nd_array = np.array(normal_list)
print(f"Normal list: {normal_list}, n D Array: {nd_array}")
print(f"Normal list: {type(normal_list)}, n D Array: {type(nd_array)}")
```

5] ✓ 0.0s

```
Normal list: [1, 2, 3, 4, 5], n D Array: [1 2 3 4 5]
Normal list: <class 'list'>, n D Array: <class 'numpy.ndarray'>
```

# NumPy Basics: ndarray

- The “ndarray” class provides a lot of functionality such as:
  - Statistical functions (mean, max, min, etc...)
  - Mathematical functions (Summation, addition, subtraction, etc...)

```
nd_array.mean() # Calculates the average
✓ 0.0s
np.float64(3.0)

nd_array.max(), nd_array.min() # Gets the minimum and maximum values
✓ 0.0s
(np.int64(5), np.int64(1))

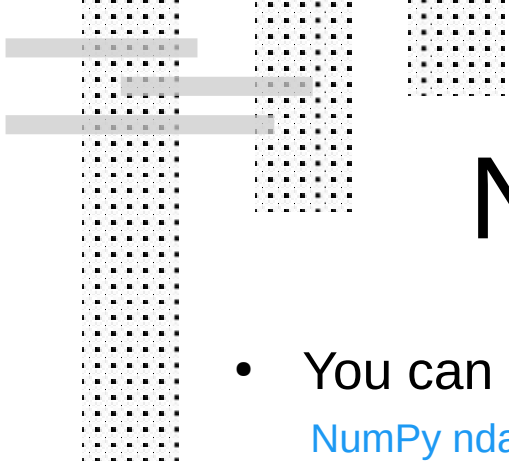
nd_array.sum() # Calculates the summation across the whole array
✓ 0.0s
np.int64(15)

nd_array + 1 # Adds one to each element in the array
nd_array - 1 # Subtracts one from each element in the array
nd_array * 1 # Multiplies each element from the array by 1
nd_array / 1 # Divides each element from the array by 1
# The output will remain the same since we did nothing
✓ 0.0s
array([1., 2., 3., 4., 5.])
```

# NumPy Basics: ndarray

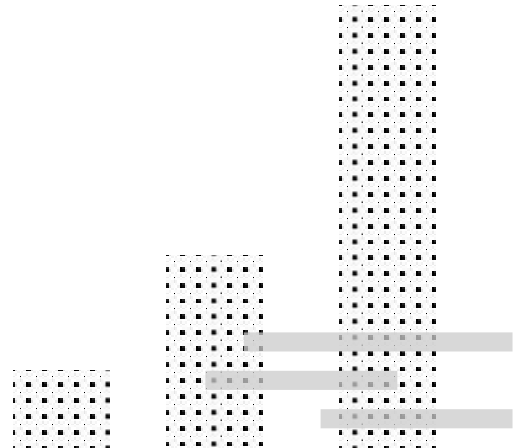
- We can also find info about the array with following:
  - The shape of the array with “arr.shape” (the shape is how many elements are in each dimension)
  - The type of the data in the array with “arr.dtype”.
  - The number of dimensions with “arr.ndim”.

```
nd_array.shape # Returns the dimensions of the array, since there is only one dimension and it has five elements, the shape is (5,)  
✓ 0.0s  
(5,)  
  
nd_array.dtype # Returns the type of the data stored in the array  
✓ 0.0s  
dtype('int64')  
  
nd_array.ndim # Returns the number of dimensions  
✓ 0.0s  
1
```



# NumPy Basics: ndarray

- You can explore more functionality at the documentation:  
[NumPy ndarray](#)



# NumPy Basics: random

- Next is generating ndarrays, using the np.random module.
  - Getting an array with random integer values within a given range is done with “np.random.randint”
  - “np.random.rand” gets random float values from a normal distribution (between 1 & 0).

```
random_int_array = np.random.randint(  
    low=1, # The lowest value that can be chosen  
    high=10, # The highest value that can be chosen  
    size=5, # the shape/length of the array  
)  
random_int_array
```

✓ 0.0s

```
array([2, 1, 7, 5, 8])
```

```
random_float_array = np.random.rand(2, 2) # takes only the shape. NOTE: the shape is NOT a tuple  
random_float_array
```

✓ 0.0s

```
array([[0.13537731, 0.03122741],  
       [0.91640602, 0.9695787 ]])
```

# NumPy Basics: random

- “np.random” also provides a lot of distributions to choose from:
  - Geometric
  - Hyper-Geometric
  - Binomial
  - etc...

```
binomial_array = np.random.binomial(n=10, p=0.5, size=(2, 4)) # Returns a Binomial Distribution random array
binomial_array
✓ 0.0s
array([[5, 6, 4, 4],
       [7, 5, 7, 8]])

geometric_array = np.random.geometric(p=0.5, size=(3, 4)) # Returns a Geometric Distribution random array
geometric_array
✓ 0.0s
array([[2, 1, 1, 4],
       [4, 1, 1, 1],
       [1, 2, 5, 1]])
```



# NumPy Basics: Matrices

- Matrices in NumPy are basically 2D ndarrays. And all the discussed matrix operations are found and more.

```
matrix= np.random.randint(low=1, high=10, size=(2, 2))
matrix
✓ 0.0s
array([[7, 1],
       [7, 7]])

matrix + 2
✓ 0.0s
array([[9, 3],
       [9, 9]])

matrix * 3
✓ 0.0s
array([[21, 3],
       [21, 21]])

matrix - 4
✓ 0.0s
array([[3, -3],
       [3, 3]])

matrix + matrix # Each element is added to the cooresponding on in the other matrix
✓ 0.0s
array([[14, 2],
       [14, 14]])
```

# NumPy Basics: Matrices

- In NumPy, since everything is an ndarray, they allow operations between incompatible size arrays:

```
one_d_array = np.random.randint(low=1, high=10, size=5)
one_d_array
✓ 0.0s
array([7, 8, 2, 3, 3])

two_d_array = np.random.randint(low=1, high=10, size=(5, 5))
two_d_array
✓ 0.0s
array([[8, 3, 4, 8, 2],
       [4, 7, 2, 2, 1],
       [1, 6, 9, 4, 7],
       [1, 3, 7, 4, 4],
       [3, 1, 1, 1, 7]])

two_d_array + one_d_array
✓ 0.0s
array([[15, 11, 6, 11, 5],
       [11, 15, 4, 5, 4],
       [ 8, 14, 11, 7, 10],
       [ 8, 11, 9, 7, 7],
       [10, 9, 3, 4, 10]])

two_d_array * one_d_array
✓ 0.0s
array([[56, 24, 8, 24, 6],
       [28, 56, 4, 6, 3],
       [ 7, 48, 18, 12, 21],
       [ 7, 24, 14, 12, 12],
       [21, 8, 2, 3, 21]])
```

# NumPy Basics: Matrices

- This is similar to scalar operations, but instead of adding a singular value to each element, it adds the whole array to each sub array of the same size:

$$\begin{array}{c} [1, 2, 3, 4, 5] \left\{ \begin{array}{l} + \rightarrow \\ + \rightarrow \\ + \rightarrow \\ + \rightarrow \end{array} \right. \left[ \begin{array}{c} [1, 2, 3, 4, 5] \\ [1, 2, 3, 4, 5] \\ [1, 2, 3, 4, 5] \\ [1, 2, 3, 4, 5] \end{array} \right] \rightarrow \left[ \begin{array}{c} [2, 4, 6, 8, 10] \\ [2, 4, 6, 8, 10] \\ [2, 4, 6, 8, 10] \\ [2, 4, 6, 8, 10] \end{array} \right] \end{array}$$

# NumPy Basics: Matrices

- However if the size of the sub-array is incompatible with the added array it throws an error:

```
wrong_size_array = np.random.randint(low=1, high=10, size=6)
wrong_size_array
✓ 0.0s
array([3, 1, 8, 9, 5, 7])

two_d_array + wrong_size_array # Throws an error due to incompatible sizes
✗ 0.0s

-----
ValueError                                Traceback (most recent call last)
Cell In[66], line 1
----> 1 two_d_array + wrong_size_array # Throws an error due to incompatible sizes

ValueError: operands could not be broadcast together with shapes (5,5) (6,)
```

# NumPy Basics: Matrices

- There is also matrix-wise multiplication, it can be done with two ways:
  - With a new operator '@'
  - With ".dot" function

```
arr1 @ arr2 # matrix of size 2 x 3 with 3 x 4 => a matrix of size 2 x 4
✓ 0.0s
array([[63, 69, 66, 46],
       [42, 28, 27, 16]])

arr1.dot(arr2) # same result
✓ 0.0s
array([[63, 69, 66, 46],
       [42, 28, 27, 16]])

(arr1 @ arr2) == (arr1.dot(arr2)) # sanity check
✓ 0.0s
array([[ True,  True,  True,  True],
       [ True,  True,  True,  True]])
```

# NumPy Basics: Matrices

- And when we try to multiply incompatible sizes, we get an error as expected:

```
wrong_size_array @ arr2 # sizes 5 x 5 and 3 x 4 are incompatible => error
✖ 0.0s

-----
ValueError                                Traceback (most recent call last)
Cell In[76], line 1
----> 1 wrong_size_array @ arr2 # sizes 5 x 5 and 3 x 4 are incompatible => error

ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with gufunc signature (n?,k),(k,m?)->(n?,m?) (size 3 is different from 5)
```

# Session Task & What's Next

- Your Task: task.ipynb in session 2 folder
- Next Session: Data Handling & Visualization with Pandas



Thank you.

